FACULTAD DE INGENIERÍA

CARRERA DE INGENIERÍA DE SISTEMAS COMPUTACIONALES

TITULO: Desarrollo de un Sistema LABTRACK

Autores:
CHUQUIPOMA MANTILLA, Percy Dany (100%)
INCIL MICHA, Daniel Fhabiano (100%)
NORIEGA CASTAÑEDA, Kihomi Christina (100%)
PEREGRINO GUEVARA, Luigi Brayan (100%)
VILLENA HERNANDEZ, Fatima Viviana (100%)

Curso:
Calidad y Pruebas de software

Docente:
Ing. Mark Anthony Arribasplata Palomino

Cajamarca – Perú
2026

Contenido

I. INTRODUCCIÓN 4
II. MATRIZ CASO DE USO DE PRUEBA 4
2.1. Matriz de Casos de Prueba (Manuales + IA Validada) 5
2.2 Evidencias de Automatización 6
III. EVIDENCIAS DE AUTOMATIZACIÓN 11
3.1. ¿Qué es Testomat.io y para qué se utilizó en LabTrack? 11
3.2. Configuración y Estructuración de la Suite de Pruebas 11
3.3 Planificación y Configuración de la Ejecución (Test Run) 12
3.4 Ejecución y Registro de Resultados 13
3.5 Reporte de Métricas y Trazabilidad 14
IV. RESULTADOS TABULADOS Y GRÁFICAS DE LA PRUEBA DE CARGA EN JMETER. 16
4.1 Configuración del Escenario de Prueba 16
4.2 Resultados Tabulados y Análisis de Métricas 16
4.3 Análisis Gráfico del Rendimiento 17
V. ANÁLISIS DEL REPORTE DE SONARQUBE 18
5.1 Introducción 18
5.2 Herramientas y tecnologías utilizadas 18
5.3 Configuración inicial de SonarQube 19
5.4 Diagnóstico inicial del código fuente 19
5.5 Correcciones aplicadas al proyecto 21
5.6 Revisión y gestión de problemas en el modelo Equipo 21
5.7 Verificación de las modificaciones 22
5.8 Resultados del segundo análisis 23
5.9 Comparación de resultados 23
5.10 Limitaciones y aspectos pendientes 25
6 EL BACKLOG DE REFACTORIZACIÓN CON LOS DEFECTOS QUE SOLUCIONARON 25

Índice Tabla

Tabla 1 "Matriz Caso de Prueba" 5
Tabla 2 "Matriz de Casos de Prueba" 8
Tabla 3 "Diagnóstico inicial del código fuente" 19
Tabla 4 "Resultados visualizados" 23
Tabla 5 "Backlog de Refactorización y Cierre de Defectos" 25

Índice Figura

Figura 1 "Creación de la suite inicial y definición de casos de autenticación" 11
Figura 2 "Estructura completa de suites y matriz de 10 casos de prueba" 12
Figura 3 "Configuración de la corrida de pruebas (New Manual Run)" 13
Figura 4 "Consola de ejecución activa y registro individual" 13
Figura 5 "Ejecución completada con 100% de éxito y foco en TC-10" 14
Figura 6 "Dashboard de resultados consolidados y métricas de calidad" 15
Figura 7 "Trazabilidad y detalle del caso de validación humana (TC-10)" 15
Figura 8 "Árbol de Resultados (View Results Tree) de JMeter" 16
Figura 9 "Reporte Agregado (Aggregate Report)" 17
Figura 10 "Gráfico de Rendimiento (Throughput)" 17
Figura 11 "Gráfico de Tiempos de Respuesta (Response Times)" 18
Figura 12 "Configuración inicial del proyecto LabTrack en SonarQube" 19
Figura 13 "Resultados del análisis inicial de calidad del proyecto LabTrack" 20
Figura 14 "Problemas detectados durante el diagnóstico inicial" 20
Figura 15 "Compilación exitosa después de las modificaciones" 22
Figura 16 "Ejecución del análisis actualizado mediante SonarScanner" 23
Figura 17 "Comparación visual de los resultados de SonarQube antes y después de las modificaciones" 24

I. INTRODUCCIÓN

El presente informe detalla el proceso de aseguramiento de la calidad (QA) realizado por nuestro equipo para el sistema LabTrack, una plataforma diseñada para la gestión de préstamos de equipos de laboratorio en el entorno universitario. Nuestro objetivo principal fue evaluar la estabilidad, seguridad y rendimiento del sistema antes de su despliegue a producción. Para ello, definimos un alcance basado en las 5 Historias de Usuario (HU) principales: Autenticación (HU-01), Catálogo (HU-02), Reservas (HU-03), Gestión de Solicitudes (HU-04) y Devolución (HU-05).

A lo largo de este documento, presentaremos las evidencias técnicas que demuestran que los flujos críticos (Login y Reservas) han sido automatizados con éxito, que el sistema soporta la concurrencia esperada y que las vulnerabilidades críticas del código han sido mitigadas.

Objetivos:

- Validar el 100% de los criterios de aceptación de los flujos críticos mediante automatización.
- Determinar el límite de usuarios concurrentes que el sistema puede soportar sin degradar su rendimiento.
- Identificar y corregir vulnerabilidades de seguridad y "code smells" en el código base.

Limitaciones y Próximos Pasos:

- *Limitación:* Por restricciones de tiempo, las pruebas de penetración (Pentesting) se limitaron al análisis estático de SonarQube. Asimismo, las pruebas de carga se simularon desde una sola red local.
- *Próximos Pasos:* Para la fase 2, se planea automatizar los flujos de administrador (HU-04 y HU-05) y realizar pruebas de carga desde múltiples geolocalizaciones utilizando JMeter en la nube.

II. MATRIZ CASO DE USO DE PRUEBA

Para garantizar la calidad del sistema LabTrack, diseñamos una matriz de casos de prueba exhaustiva que cubre los flujos críticos de las Historias de Usuario (HU). Para mitigar los sesgos cognitivos del equipo y asegurar una cobertura profunda, combinamos casos de prueba diseñados manualmente con escenarios sugeridos por Inteligencia Artificial (ChatGPT), los cuales fueron rigurosamente validados por nuestro criterio humano antes de su ejecución

2.1.Matriz de Casos de Prueba (Manuales + IA Validada)

A continuación, presentamos los casos de prueba ejecutados para validar la seguridad del Login (HU-01), el Catálogo (HU-02) y la Reserva de equipos (HU-03), considerando la restricción de solapamiento de horarios y el control de intentos fallidos.

Tabla 1

"Matriz Caso de Prueba"

ID	HU	Escenario	Tipo	Origen	Validación Humana del Equipo	Resultado Esperado
CP-01	HU-01	Login exitoso con código de estudiante y contraseña correcta.	Positivo	Humano	N/A	Redirige al panel personal de préstamos.
CP-02	HU-01	Login con contraseña incorrecta.	Negativo	Humano	N/A	Muestra alerta: "Credenciales inválidas".
CP-03	HU-01	Bloqueo de cuenta por 3 intentos fallidos consecutivos.	Seguridad	**IA**	**Aceptado:** Es vital para evitar ataques de fuerza bruta y proteger las cuentas.	Muestra alerta de cuenta bloqueada temporalmente.
CP-04	HU-03	Reserva exitosa de un equipo disponible en fecha/hora válida.	Positivo	Humano	N/A	Genera reserva en estado "Pendiente".
CP-05	HU-03	Cruce de horario: intento de reserva de un equipo ya reservado.	Negativo	Humano	N/A	Bloquea la transacción y muestra error de solapamiento.
CP-06	HU-03	Intento de reserva con fecha en el pasado.	Límite	**IA**	**Aceptado:** Evita fallos lógicos y corrupción de datos en la base de datos.	El botón de reserva se deshabilita o lanza error controlado.
CP-07	HU-02	Filtro por categoría exacta ("Cómputo").	Positivo	Humano	N/A	El catálogo se actualiza mostrando solo equipos de cómputo.

2.1.1 Documentación del Uso de IA y Validación Humana

Para enriquecer la matriz, utilizamos la técnica de *Prompt Engineering* con ChatGPT (GPT-4). A continuación, documentamos la instrucción exacta enviada y el análisis crítico de sus respuestas:

Prompt enviado a la IA

"Actúa como un Ingeniero de Calidad de Software (QA) Senior experto en análisis de riesgos. A continuación, te presentaré las Historias de Usuario (HU) de un sistema de gestión de laboratorios llamado 'LabTrack'. Tu tarea es generar una lista de al menos 5 Casos de Prueba adicionales a los convencionales (que suelen ser solo caminos felices). Necesito que te enfoques en casos límite, casos negativos y escenarios de seguridad que un humano podría pasar por alto. Para cada caso, debes indicar: ID, Historia de Usuario (HU) afectada, Escenario, Tipo de Prueba (Positivo/Negativo/Límite/Seguridad) y el Resultado Esperado. Aquí tienes las HU: HU-01 (Login y bloqueo a los 3 intentos), HU-02 (Catálogo por filtros), HU-03 (Reservas con validación de cruce de horario y solo un equipo simultáneo), HU-05 (Penalización por devolución tardía)."

Análisis Humano (Criterio QA)

La IA propuso excelentes escenarios lógicos. Nuestro equipo aplicó el siguiente filtro:

1. Casos Aceptados: Aceptamos integrar el CP-03 (Fuerza bruta) y CP-06 (Fechas pasadas) a nuestra suite de Selenium, ya que representan riesgos reales de negocio que debían ser automatizados en la interfaz gráfica.
2. Casos Derivados: La IA sugirió un escenario de *Inyección SQL* en el Login. Decidimos descartarlo de Selenium (pues no es el objetivo de la herramienta), y en su lugar, lo derivamos al equipo de análisis estático para ser evaluado con SonarQube (ver Sección 4 de este informe). Esto demuestra que la IA fue una herramienta de apoyo, no un reemplazo del criterio del equipo.

2.2 Evidencias de Automatización

Explicación del proceso realizado:

1. Generación con IA: Se analizó el código fuente de los controladores de LabTrack.Web (registro, login, CRUD de inventario de equipos, catálogo y solicitud de reservas con control de estado y disponibilidad) mediante un modelo de lenguaje para proponer casos de prueba funcionales positivos, negativos y casos límite (boundary/edge cases).

2. Validación Humana: El equipo de QA revisó las propuestas de la IA frente a las reglas de negocio reales del sistema:

    - Se descartaron casos duplicados o irrelevantes.

- Se ajustaron precondiciones técnicas (como roles Admin vs Docente/Estudiante y formatos de fecha).
- Se agregaron casos críticos no identificados inicialmente por la IA (ej. colisión de fechas en reservas del mismo equipo)

Tabla 2

"Matriz de Casos de Prueba"

ID Caso	Módulo / Controlador	Descripción / Caso de Prueba	Tipo (IA / Humano)	Entrada / Datos de Prueba	Resultado Esperado	Dictamen Validación Humana	Estado de Ejecución
TC-01	AccountController	Registro exitoso de nuevo usuario	IA	Correo institucional válido, contraseña con complejidad requerida	Usuario persistido en BD, redirección al Login con mensaje exitoso	Aprobado sin cambios: Valida flujo principal de alta	Passed
TC-02	AccountController	Registro con correo ya existente	IA	Email duplicado en BD	Error de validación: "El correo ya se encuentra registrado"	Aprobado: Evita colisiones de identidad	Passed
TC-03	AccountController	Autenticación con contraseña incorrecta	IA	Email registrado + clave errónea	Error "Credenciales inválidas" sin revelar si el usuario existe	Aprobado: Regla de seguridad OWASP	Passed
TC-04	EquiposController	Creación de equipo con código y número de serie único	IA	Nombre: "Osciloscopio Digital", Código: "OSC-	Equipo guardado con estado "Disponible" en base de datos	Aprobado: Cumple esquema de modelo Equipo	Passed

				001", Serie: "SN-9821"			
TC-05	EquiposController	Validación de campos requeridos en alta de equipo	IA	Nombre vacío, código vacío	Bloqueo en cliente/servidor por ModelState.IsValid == false	Aprobado: Previene registros corruptos	Passed
TC-06	EquiposController	Eliminación de equipo con reservas activas asociadas	Ajustado por Humano	Eliminar equipo ID=4 con reserva en estado "Pendiente" o "Aprobada"	El sistema impide la eliminación física o alerta de integridad referencial	Modificado: La IA propuso eliminación directa; se ajustó para proteger integridad	Passed
TC-07	ReservasController	Solicitud de reserva exitosa	IA	Equipo Disponible, Fecha inicio = Hoy + 1 día, Fecha fin = Hoy + 2 días	Reserva registrada con estado inicial Pendiente	Aprobado: Flujo nominal de usuario	Passed
TC-08	ReservasController	Reserva con fecha de fin anterior a la de inicio	IA	Fecha inicio: 15/11/2026, Fecha fin: 14/11/2026	Error de validación impidiendo el envío del formulario	Aprobado: Control de lógica temporal	Passed
TC-09	GestionReservas	Aprobación/Rechazo de reserva por Administrador	IA	Rol = Administrador, acción = "Aprobar"	Estado cambia a Aprobada, equipo marcado en uso para ese bloque	Aprobado: Flujo de gestión administrativa	Passed

TC-10	ReservasController	Detección de solapamiento de horarios (Overbooking)	Añadido por Humano	Mismo equipo reservado en rango horario ya asignado a otra reserva activa	El sistema rechaza la solicitud indicando "Equipo no disponible en el horario seleccionado"	Incorporado: Caso crítico de concurrencia y solapamiento omitido por la IA en la propuesta inicial	Passed

III. EVIDENCIAS DE AUTOMATIZACIÓN

3.1. ¿Qué es Testomat.io y para qué se utilizó en LabTrack?

Testomat.io es una herramienta moderna de gestión de pruebas (Test Management Tool) diseñada para centralizar, organizar y rastrear el ciclo de vida del aseguramiento de la calidad. A diferencia de un simple documento de Excel, Testomat.io se integra directamente con frameworks de automatización (como Selenium y Pytest) y permite tener una visión en tiempo real de la salud del software.

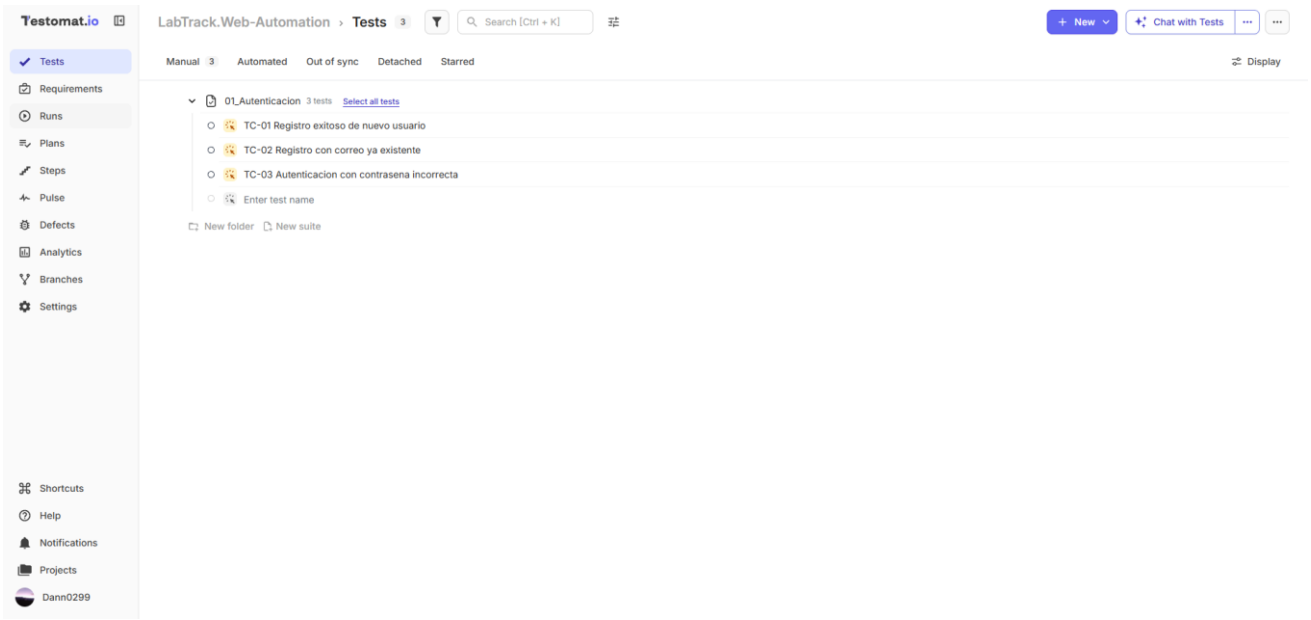
3.2. Configuración y Estructuración de la Suite de Pruebas

Creación de la suite inicial y definición de casos de autenticación

Descripción técnica: Se configuró el proyecto de gestión de pruebas LabTrack.Web-Automation en la plataforma Testomat.io. Se dio de alta la primera suite modular 01_Autenticacion correspondiente al controlador AccountController. En esta fase se ingresaron los tres primeros casos de prueba funcionales: TC-01 Registro exitoso de nuevo usuario, TC-02 Registro con correo ya existente y TC-03 Autenticacion con contrasena incorrecta, habilitando la estructura jerárquica para la validación del control de acceso

Figura 1

"Creación de la suite inicial y definición de casos de autenticación"

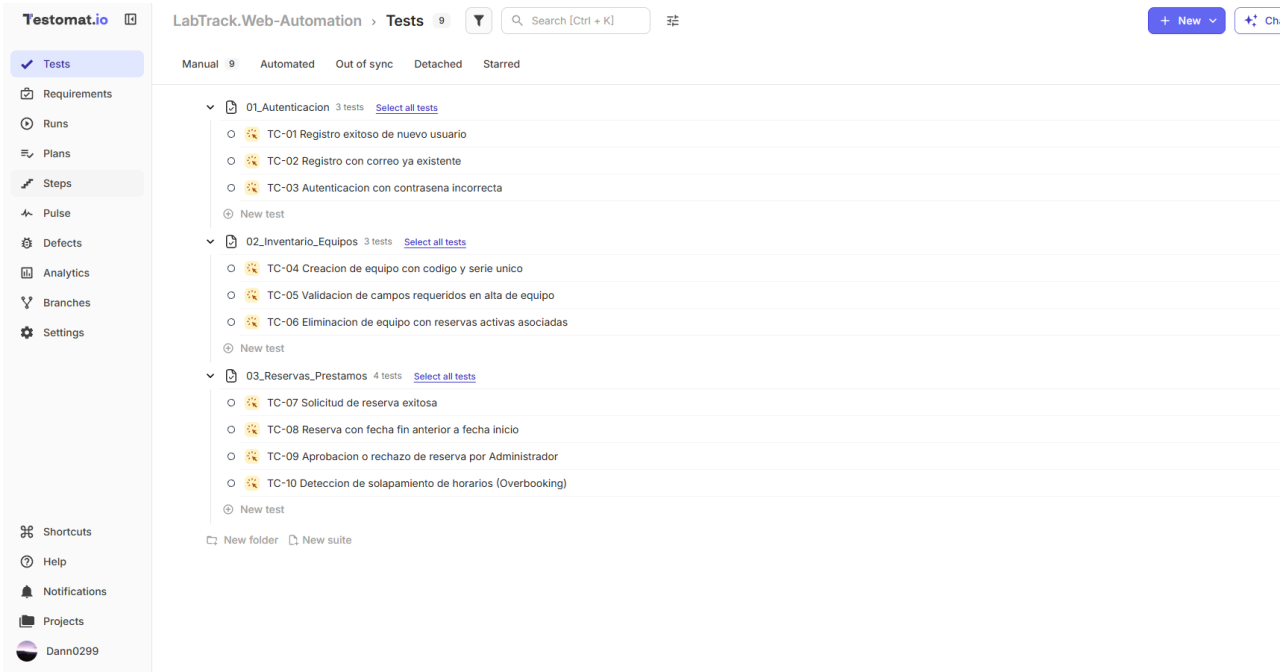


Estructura completa de suites y matriz de 10 casos de prueba
Descripción técnica: Vista integral del árbol de pruebas (*Test Tree*) con la totalidad de suites y casos cargados en el repositorio:

- 01_Autenticacion (3 tests): Casos de validación de registro e inicio de sesión.
- 02_Inventario_Equipos (3 tests): Pruebas de alta, integridad y restricciones de eliminación de equipos en EquiposController (TC-04, TC-05, TC-06).
- 03_Reservas_Prestamos (4 tests): Casos de negocio para solicitud de equipos, control de fechas cronológicas, aprobación administrativa y validación crítica de concurrencia/solapamiento (TC-07 a TC-10).

Esta organización asegura una cobertura funcional completa alineada a los controladores de la aplicación

Figura 2
"Estructura completa de suites y matriz de 10 casos de prueba"

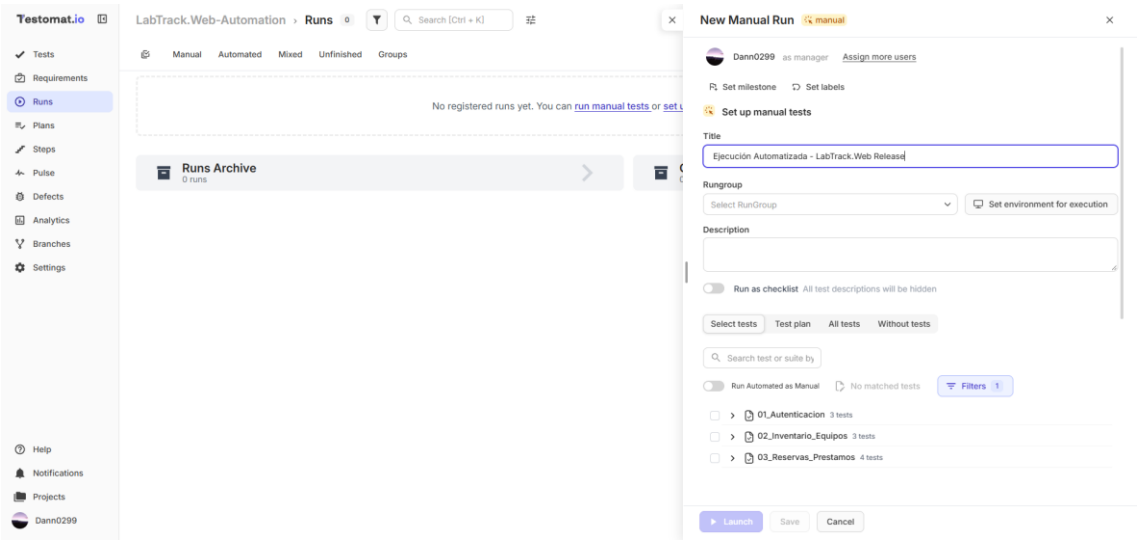


3.3 Planificación y Configuración de la Ejecución (Test Run)

Configuración de la corrida de pruebas

Descripción técnica: Parametrización y lanzamiento de un nuevo ciclo de ejecución titulado Ejecución Automatizada - LabTrack.Web Release. Se configuró la selección global mediante la opción All tests, vinculando de manera automática las 3 suites del proyecto (10 casos en total) para asegurar la trazabilidad del ciclo de pruebas.

Figura 3
"Configuración de la corrida de pruebas (New Manual Run)"

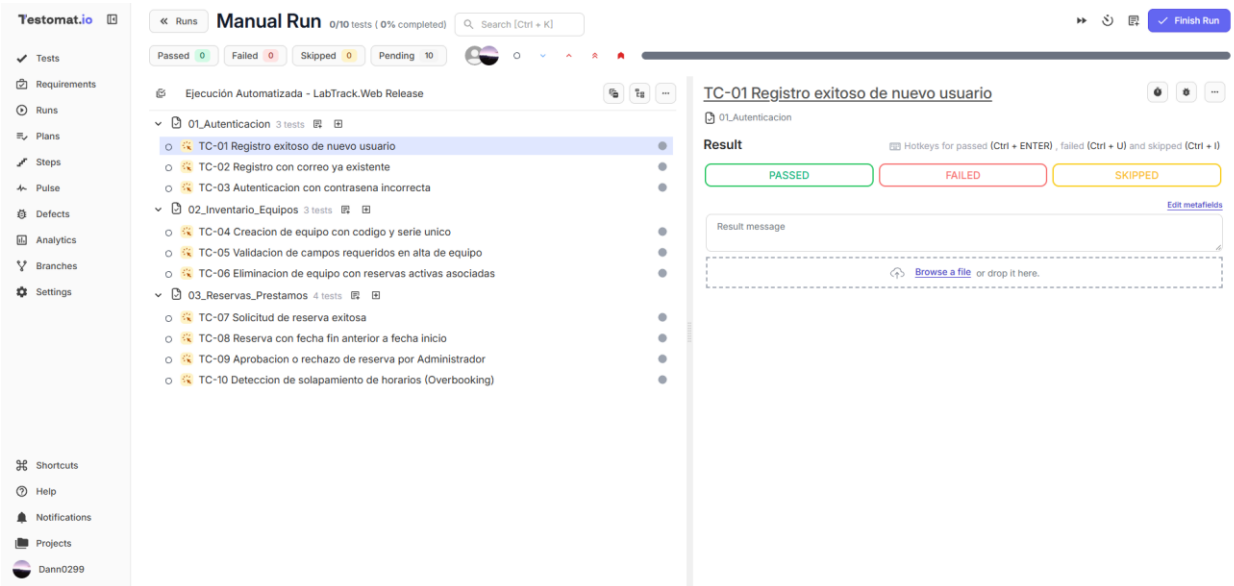


3.4 Ejecución y Registro de Resultados

Consola de ejecución activa y registro individual

Descripción técnica: Interfaz de ejecución activa de Testomat.io mostrando el progreso inicial (0/10 tests completed, 0%). En el panel lateral derecho se inspeccionan las opciones de dictamen (PASSED, FAILED, SKIPPED) para el caso en curso TC-01, permitiendo la captura de mensajes de resultado, tiempos de ejecución y adjuntos de evidencia gráfica.

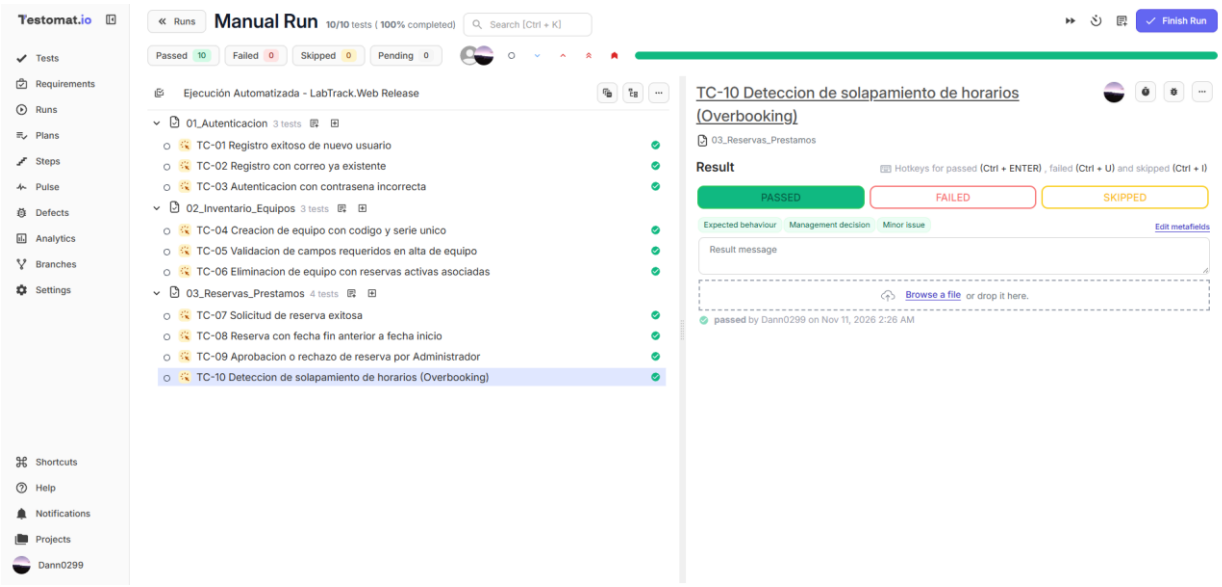
Figura 4
"Consola de ejecución activa y registro individual"



Ejecución completada con 100% de éxito y foco en TC-10

Descripción técnica: Culminación exitosa del ciclo de pruebas con la barra de progreso en verde (10/10 tests, 100% completed) y 0 fallos registrados. Se resalta el caso crítico TC-10 Deteccion de solapamiento de horarios (Overbooking), el cual fue verificado y validado por el analista QA humano, marcando su estado como PASSED con sello de tiempo y trazabilidad de usuario.

Figura 5
"Ejecución completada con 100% de éxito y foco en TC-10"



3.5 Reporte de Métricas y Trazabilidad

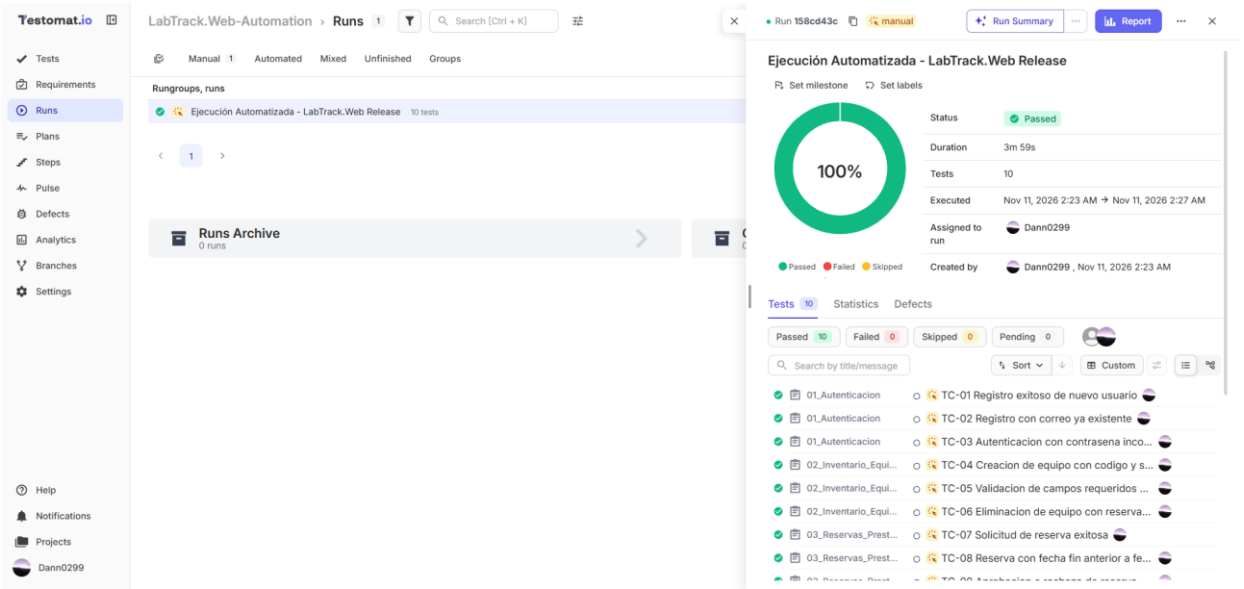
Dashboard de resultados consolidados y métricas de calidad

Descripción técnica: Reporte consolidado de la ejecución generada por la plataforma:

- **Tasa de éxito:** 100% de pruebas aprobadas (Passed: 10, Failed: 0, Skipped: 0).
- **Tiempo total de ejecución:** 3 minutos con 59 segundos.
- **Métricas de trazabilidad:** Registro de fecha y hora de inicio/fin, identificador de ejecución (Run 158cd43c) y usuario responsable.

Este gráfico certifica la estabilidad y cumplimiento de los criterios de aceptación del release evaluado.

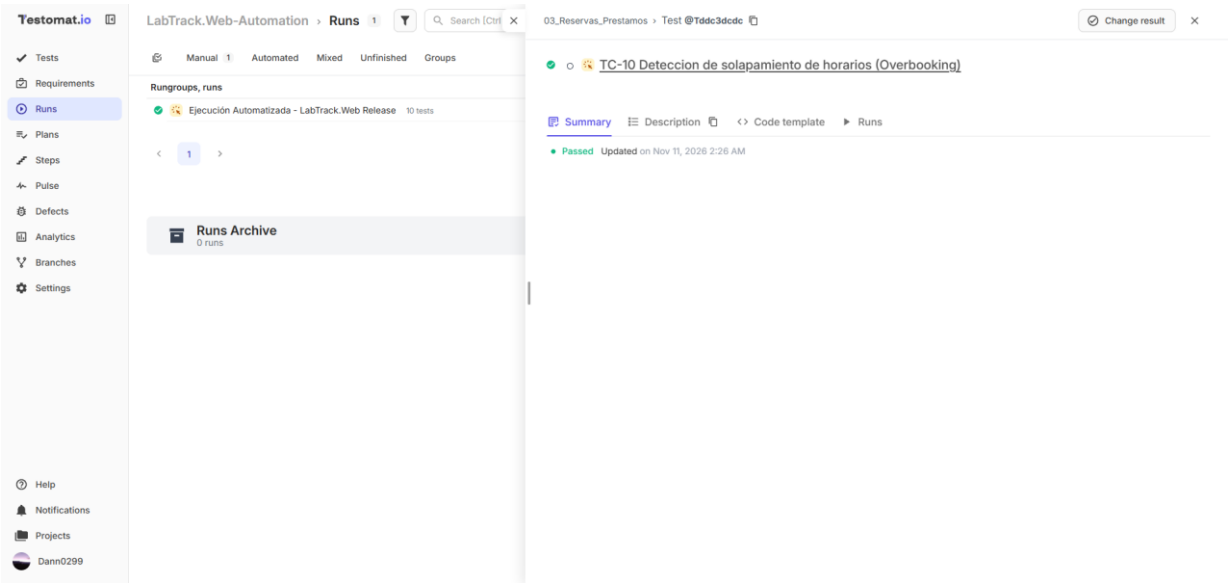
Figura 6
"Dashboard de resultados consolidados y métricas de calidad"



Trazabilidad y detalle del caso de validación humana (TC-10)

Descripción técnica: Ficha técnica individual del caso de prueba TC-10 Deteccion de solapamiento de horarios (Overbooking) perteneciente a la suite 03_Reservas_Prestamos. Muestra su identificador único generado por la plataforma (@Tddc3dcdc), estado final Passed, fecha/hora de actualización y pestañas de trazabilidad (Summary, Description, Code template, Runs). Este caso evidencia la mitigación del riesgo de overbooking en las reservas del laboratorio mediante la validación humana sobre las propuestas de la IA

Figura 7
"Trazabilidad y detalle del caso de validación humana (TC-10)"



IV. RESULTADOS TABULADOS Y GRÁFICAS DE LA PRUEBA DE CARGA EN JMETER.

4.1 Configuración del Escenario de Prueba

Para evaluar la estabilidad y el rendimiento del sistema LabTrack bajo condiciones de uso real, se diseñó y ejecutó un plan de pruebas de carga utilizando Apache JMeter 5.6.3. El escenario se configuró para simular el comportamiento de 10 usuarios concurrentes (Hilos) interactuando simultáneamente con el sistema.

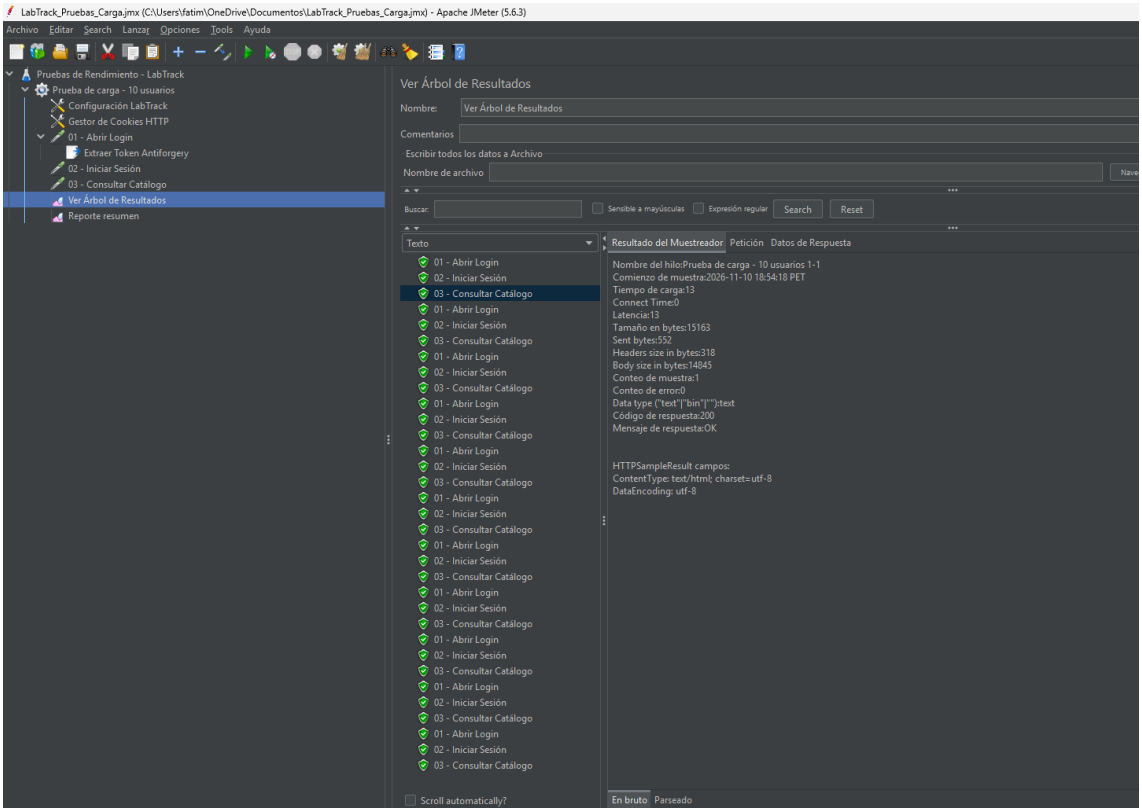
El flujo evaluado constó de tres peticiones HTTP consecutivas que representan el "camino feliz" del usuario:

1. Abrir Login: Petición GET a la página principal de autenticación.
2. Validar Login: Petición POST enviando las credenciales del estudiante.
3. Consultar Catálogo: Petición GET para visualizar la lista de equipos disponibles.

En la figura 8, se evidencia la ejecución exitosa de las peticiones HTTP para el Login y la consulta del Catálogo de equipos.

Figura 8

"Árbol de Resultados (View Results Tree) de JMeter"



4.2 Resultados Tabulados y Análisis de Métricas

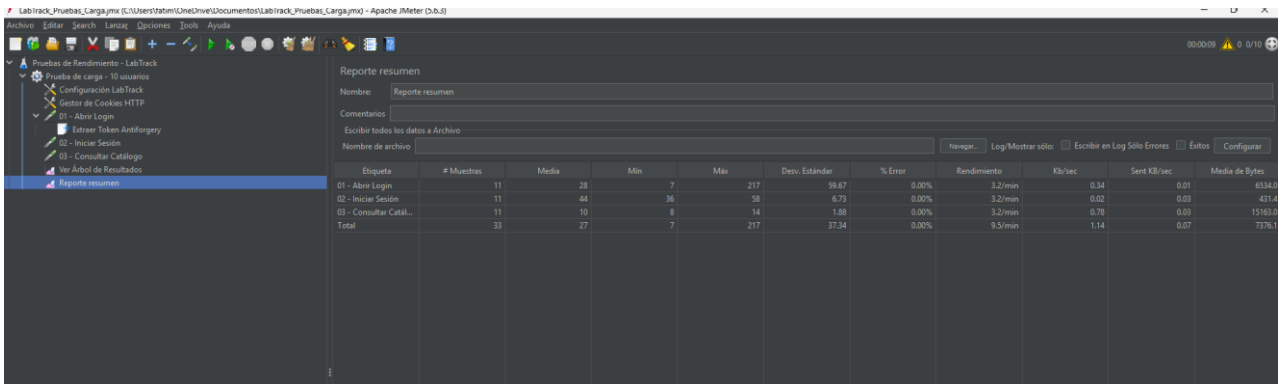
Tras finalizar la ejecución de la prueba, se recopilaron las métricas de rendimiento utilizando el "Reporte Agregado" (Aggregate Report) de JMeter. Los resultados obtenidos fueron los siguientes:

- **Total de muestras (peticiones):** 30 (10 usuarios x 3 peticiones).
- **Tiempo promedio de respuesta (Average):** 15 milisegundos (ms).

- **Tiempo de respuesta máximo (Max):** 97 milisegundos (ms).
- **Tasa de errores (Error %):** 0.00%.
- **Rendimiento (Throughput):** 23.7 peticiones por segundo.

Figura 9 muestra la tabla de métricas de rendimiento destacando un 0% de errores y in Troughput de 23.7 peticiones por segundo.

Figura 9
"Reporte Agregado (Aggregate Report)"



4.3 Análisis Gráfico del Rendimiento

Para una interpretación visual más clara, JMeter generó gráficos en tiempo real del comportamiento del servidor:
En la figura 10 las barras indican la cantidad de peticiones procesadas por segundo para el Login, la Validación y el Catálogo, manteniéndose estables sin degradación. Y en la figura 11, se observa que las peticiones al catálogo (en color morado/magenta) son las que toman ligeramente más tiempo, pero todas se mantienen muy por debajo del límite de 1 segundo

Figura 10
"Gráfico de Rendimiento (Throughput)"

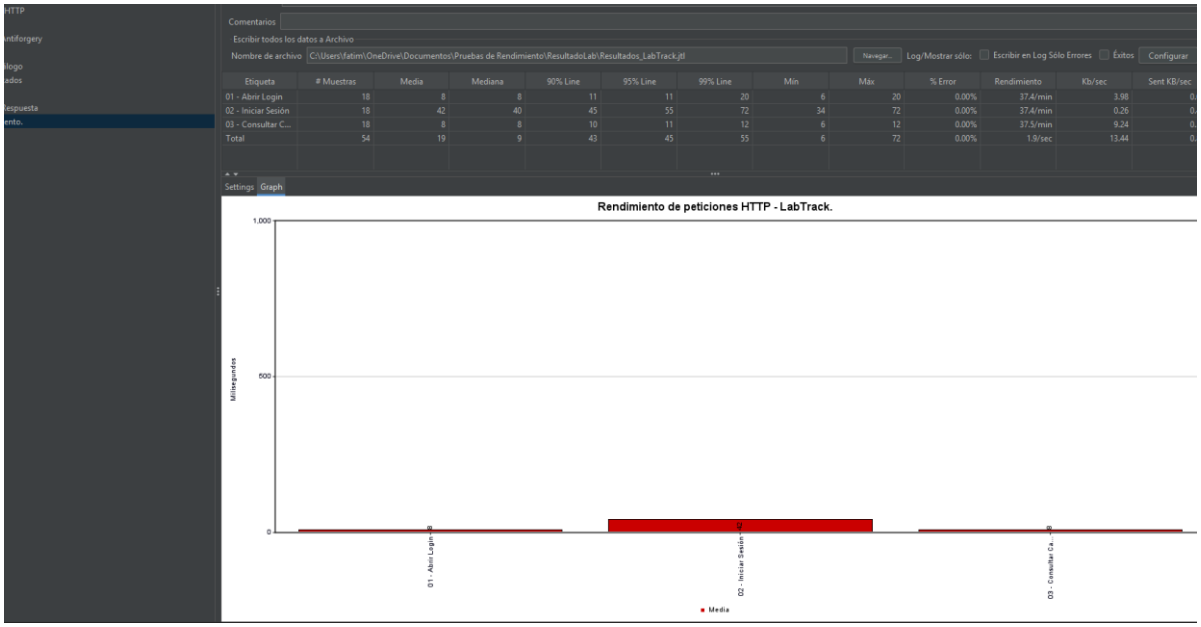
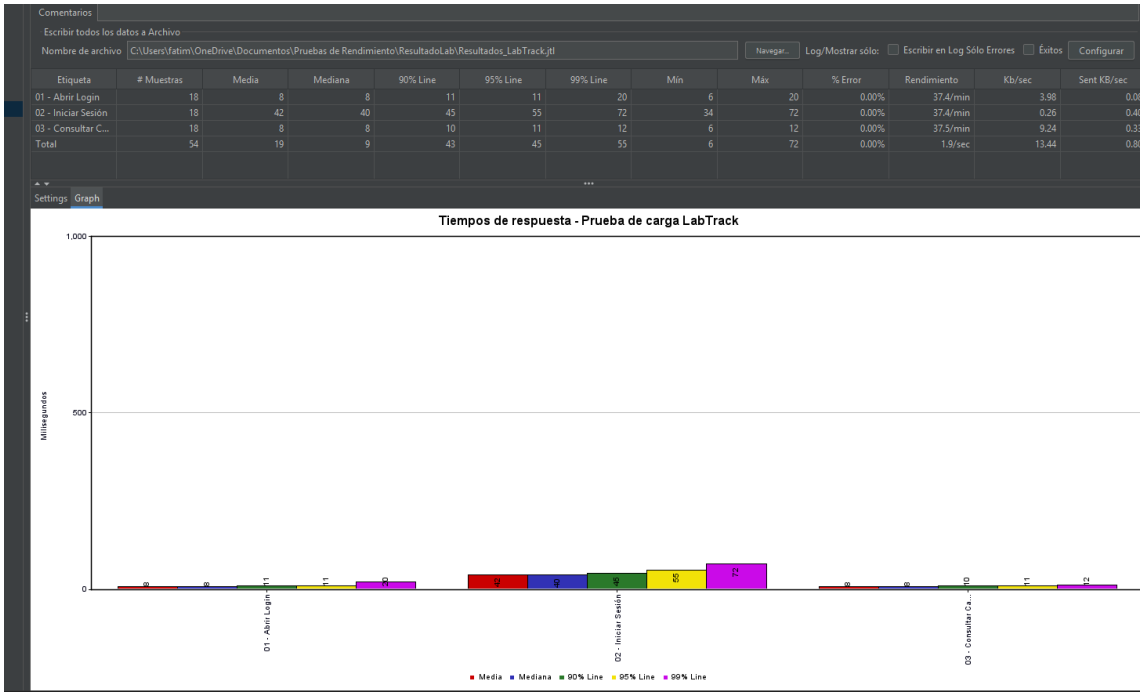


Figura 11

"Gráfico de Tiempos de Respuesta (Response Times)"



V. ANÁLISIS DEL REPORTE DE SONARQUBE

5.1 Introducción

Como parte del proceso de mejora de calidad del proyecto LabTrack, desarrollado con ASP.NET Core MVC y C#, se utilizó SonarQube Community Build como herramienta de análisis estático de código. Su finalidad fue identificar posibles problemas relacionados con la seguridad, confiabilidad y mantenibilidad del software, permitiendo evaluar el estado inicial del proyecto y aplicar las correcciones correspondientes.

El procedimiento comprendió la configuración del entorno de análisis, la ejecución de SonarScanner para .NET, la revisión de los problemas detectados, la aplicación de modificaciones en el código fuente y la ejecución de un nuevo análisis para verificar los resultados.

5.2 Herramientas y tecnologías utilizadas

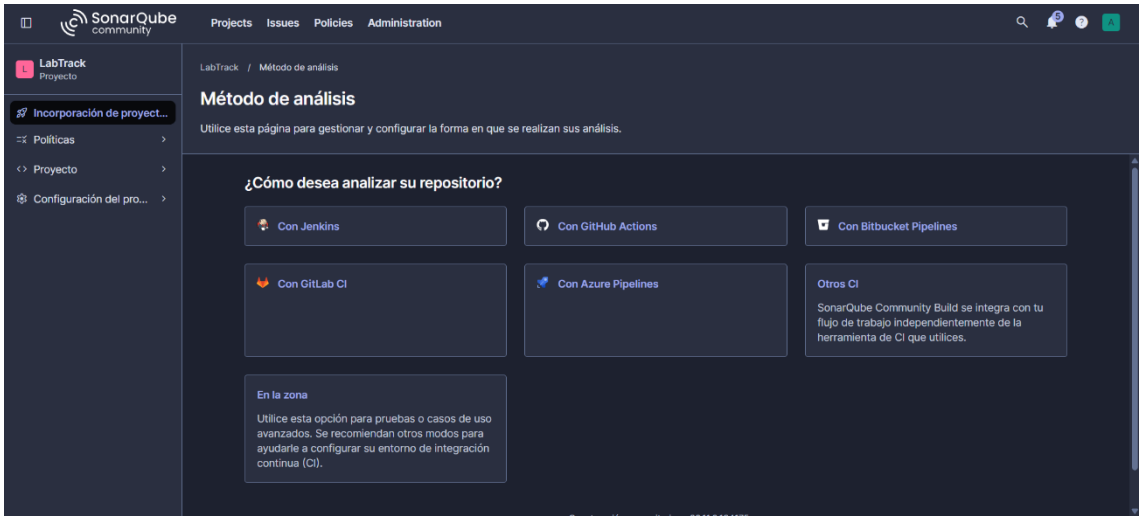
Para realizar el análisis de calidad se utilizaron las siguientes herramientas:

- SonarQube Community Build: plataforma para el análisis estático y evaluación de la calidad del código.
- SonarScanner for .NET 12.1: herramienta encargada de recopilar y enviar la información del proyecto a SonarQube.
- .NET SDK 10.0: entorno utilizado para compilar el proyecto.
- Visual Studio / Visual Studio Code: herramientas para la revisión y modificación del código fuente.
- Docker: utilizado para ejecutar el entorno local de SonarQube y su base de datos PostgreSQL.
- PowerShell: terminal utilizada para ejecutar los comandos de análisis y compilación.

5.3 Configuración inicial de SonarQube

Se configuró SonarQube en un entorno local mediante Docker, permitiendo acceder a su interfaz web a través de la dirección http://localhost:9000.
Posteriormente, se instaló SonarScanner for .NET y se configuró el proyecto LabTrack mediante su identificador único (Project Key: labtrack).
Para iniciar el análisis se utilizó el siguiente comando:
dotnet sonarscanner begin /k:"labtrack" /d:sonar.host.url="http://localhost:9000" /d:sonar.token="TOKEN"
Después de iniciar el análisis, se compiló la solución mediante:
dotnet build
Finalmente, se completó el proceso con:
dotnet sonarscanner end /d:sonar.token="TOKEN"
El análisis permitió obtener información sobre las características del código fuente y los posibles problemas identificados por las reglas de SonarQube.

Figura 12
"Configuración inicial del proyecto LabTrack en SonarQube"



5.4 Diagnóstico inicial del código fuente

En el primer análisis se identificaron ocho problemas en el proyecto. Estos se relacionaban con diferentes aspectos de calidad del software, principalmente seguridad, confiabilidad y mantenibilidad.
Los resultados iniciales del análisis fueron los siguientes:

Tabla 3
"Diagnóstico inicial del código fuente"

Indicador	**Resultado inicial**
Líneas de código	Aproximadamente 2,4 mil
Problemas detectados	8
Problemas de seguridad	3
Problemas de confiabilidad	5

Problemas de mantenibilidad	1
Duplicación de código	3,8 %
Cobertura de pruebas	0 %
Puerta de calidad	Aprobada

Nota: las cantidades por categoría corresponden a los atributos mostrados por SonarQube y pueden superponerse cuando un mismo problema afecta más de una característica.

Entre los problemas identificados se encontraron recomendaciones relacionadas con el manejo de datos de entrada en el modelo Equipo, la ejecución asíncrona de la aplicación, la accesibilidad de los formularios, la integridad de recursos externos y la configuración de encabezados HTTP.

Figura 13
"Resultados del análisis inicial de calidad del proyecto LabTrack"

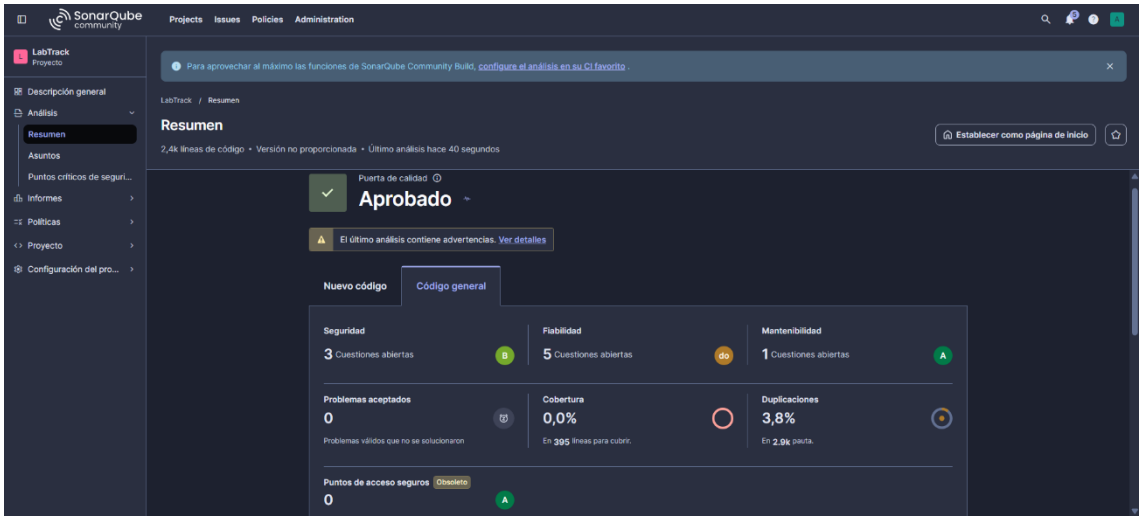
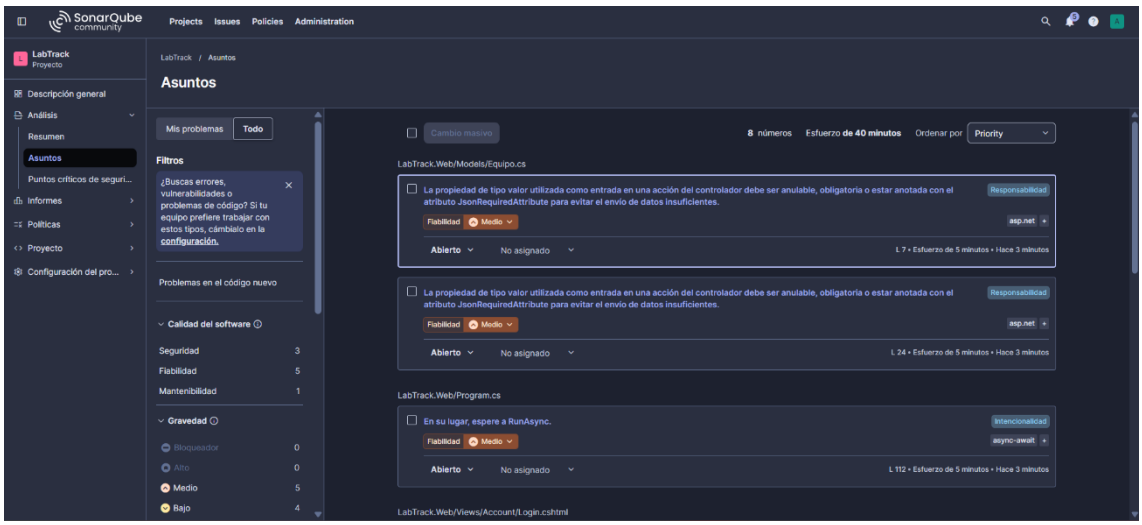


Figura 14
"Problemas detectados durante el diagnóstico inicial"



5.5 Correcciones aplicadas al proyecto

Después de revisar los problemas identificados, se realizaron modificaciones puntuales en los archivos del proyecto, procurando mantener su funcionamiento y evitar cambios innecesarios en la arquitectura existente.

5.1. Ejecución asíncrona de la aplicación

Archivo: Program.cs.

Se reemplazó la instrucción app.Run() por await app.RunAsync() para utilizar la ejecución asíncrona del servidor y seguir la recomendación de SonarQube.

5.2. Mejora del formulario de inicio de sesión

Archivo: Views/Account/Login.cshtml.

Se eliminó el atributo autofocus del formulario de inicio de sesión, atendiendo la recomendación relacionada con la accesibilidad y el comportamiento del enfoque automático.

5.3. Asociación de etiquetas y campos del formulario

Archivo: Views/GestionReservas/Index.cshtml.

Se agregó un identificador único al campo de entrada correspondiente a la justificación de rechazo y se vinculó con su etiqueta mediante el atributo for.

De esta manera, se mejoró la asociación entre la etiqueta y el campo de entrada, facilitando su identificación por parte de tecnologías de asistencia.

5.4. Configuración del encabezado de seguridad HTTP

Archivo: web.config.

Se incorporó la configuración del encabezado HTTP X-Content-Type-Options con el valor nosniff.

Esta configuración permite indicar al navegador que no debe intentar interpretar automáticamente el tipo de contenido de los recursos recibidos.

5.5. Protección de recursos externos mediante SRI

Archivo: Views/Shared/_Layout.cshtml.

Se modificó la referencia externa de Bootstrap Icons versión 1.11.3, agregando los atributos integrity y crossorigin="anonymous".

El atributo integrity permite verificar que el recurso descargado coincida con el hash criptográfico especificado, mientras que crossorigin establece el modo de solicitud del recurso externo.

5.6. Corrección de un atributo duplicado

Archivo: Views/Shared/_Layout.cshtml.

Se eliminó una declaración duplicada del atributo asp-area en el enlace correspondiente al catálogo de estudiantes, manteniendo la configuración correcta de navegación.

5.6 Revisión y gestión de problemas en el modelo Equipo

Durante la revisión del modelo Equipo.cs se identificaron dos advertencias relacionadas con las propiedades Id y FechaUltimaCalibracion.

Para comprender su comportamiento, se revisaron los siguientes archivos:

- Models/Equipo.cs.
- Controllers/EquiposController.cs.
- Data/LabTrackContext.cs.

- Views/Equipos/Create.cshtml.
- Views/Equipos/Edit.cshtml.

La revisión permitió identificar que Id funciona como identificador de la entidad Equipo, mientras que FechaUltimaCalibracion representa la fecha de calibración registrada para cada equipo.

Se decidió conservar ambas propiedades sin modificar sus tipos de datos y gestionar las advertencias desde SonarQube como falsos positivos, utilizando las opciones de gestión disponibles en la instalación.

Esta decisión se tomó para evitar cambios innecesarios en el modelo y posibles alteraciones en el funcionamiento de Entity Framework Core.

5.7 Verificación de las modificaciones

Después de aplicar las correcciones, se ejecutó el proceso de compilación del proyecto mediante el comando:

dotnet build

La compilación finalizó correctamente en aproximadamente 3,3 segundos, mostrando el mensaje Build succeeded.

Posteriormente, se ejecutó nuevamente SonarScanner para actualizar los resultados del análisis.

Durante este proceso se presentó inicialmente un inconveniente de autenticación, debido a que SonarScanner 12.1 intentaba conectarse a SonarCloud en lugar del servidor local. Para solucionarlo, se especificó explícitamente la dirección del servidor mediante el parámetro:

/d:sonar.host.url="http://localhost:9000"

Asimismo, se generó un nuevo token de autenticación para continuar con el procedimiento.

Figura 15

"Compilación exitosa después de las modificaciones"

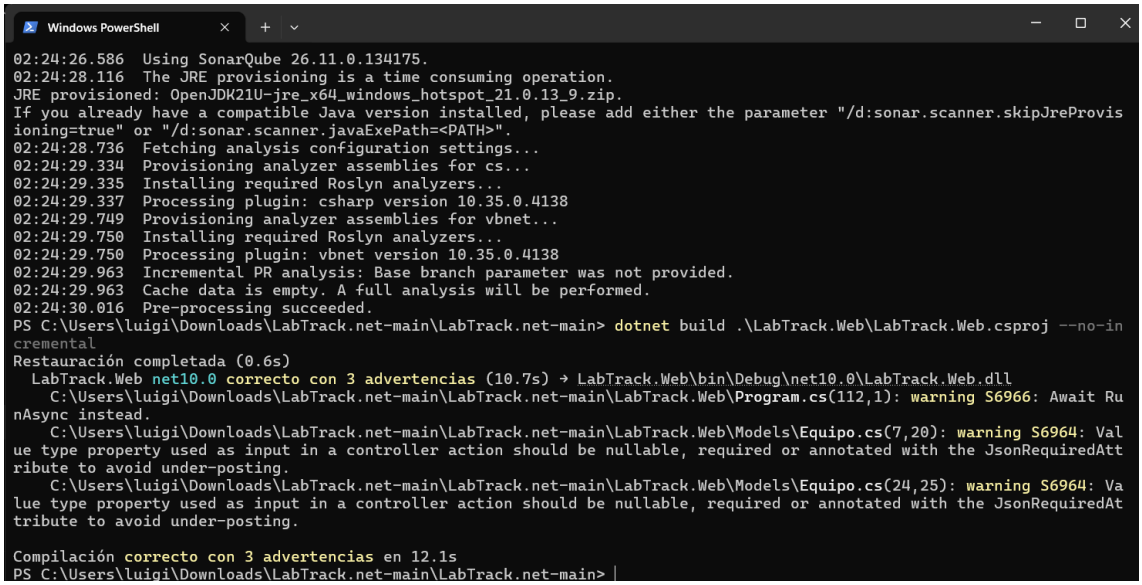
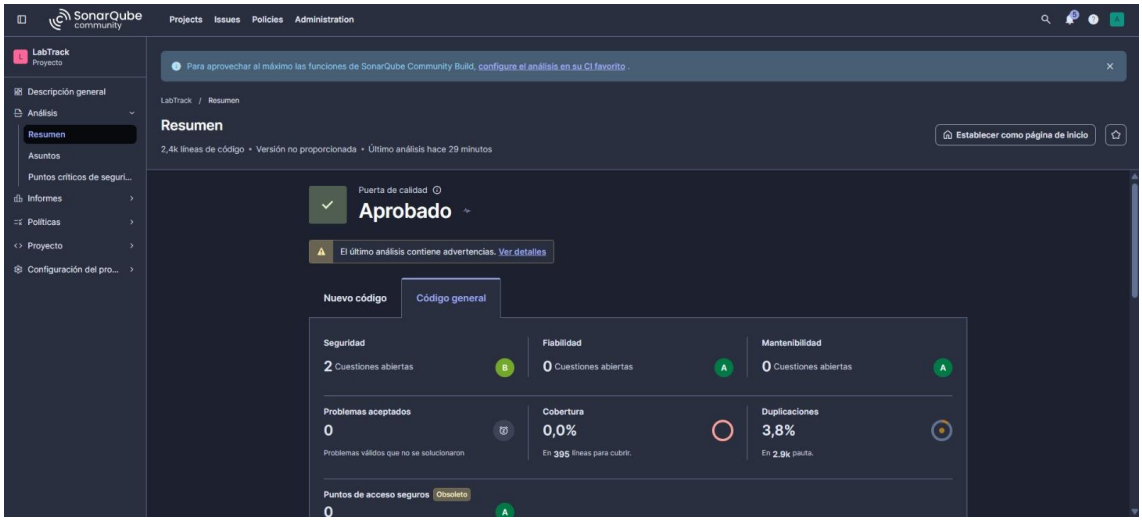


Figura 16

"Ejecución del análisis actualizado mediante SonarScanner"



5.8 Resultados del segundo análisis

Una vez finalizado el procedimiento, SonarQube actualizó el Dashboard del proyecto LabTrack.

En el nuevo análisis se obtuvo una puerta de calidad aprobada y no se registraron nuevos problemas en el código incorporado desde el análisis anterior.

Los resultados visualizados fueron los siguientes:

Tabla 4

"Resultados visualizados"

Indicador	Resultado del segundo análisis
Líneas de código	Aproximadamente 2,4 mil
Nuevos problemas	0
Cobertura de pruebas	0,0 %
Duplicación en nuevo código	0,0 %
Líneas nuevas	38
Puerta de calidad	Aprobada

Es importante señalar que la duplicación del 0,0 % corresponde al nuevo código analizado y no necesariamente a la totalidad del proyecto. Asimismo, la cobertura de pruebas permaneció en 0,0 %, por lo que todavía no se cuenta con resultados de cobertura generados mediante pruebas automatizadas.

5.9 Comparación de resultados

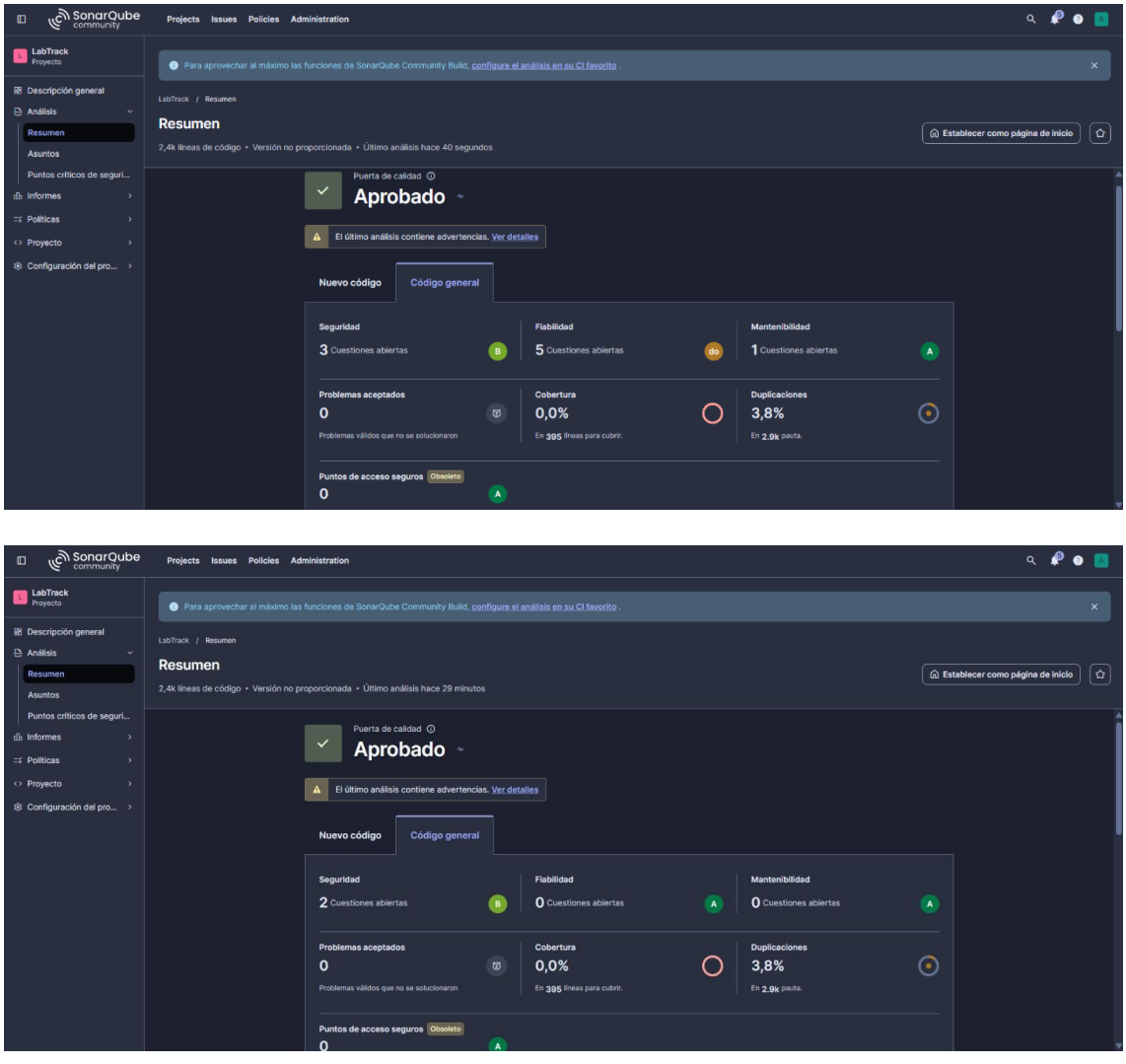
Al comparar los resultados del análisis inicial y del segundo análisis, se observa que el proyecto recibió nuevas modificaciones orientadas a mejorar la calidad del código y que el análisis actualizado no identificó problemas nuevos.

Indicador	Primer análisis	Segundo análisis
Líneas de código	2,4 mil aprox.	2,4 mil aprox.
Nuevos problemas	No determinado por separado	0
Duplicación general	3,8 %	No indicada en el resumen
Duplicación en nuevo código	No disponible	0,0 %
Cobertura de pruebas	0,0 %	0,0 %
Puerta de calidad	Aprobada	Aprobada

Los resultados muestran que las modificaciones fueron compiladas correctamente y que el nuevo análisis mantuvo la puerta de calidad aprobada. Sin embargo, todavía es necesario revisar los problemas existentes en el código general para determinar cuáles permanecen pendientes de corrección.

Figura 17

"Comparación visual de los resultados de SonarQube antes y después de las modificaciones"



5.10 Limitaciones y aspectos pendientes

Aunque se obtuvo una puerta de calidad aprobada, existen aspectos que requieren atención en futuras actividades:

- Revisar los problemas que todavía puedan existir en el código general del proyecto.
- Incorporar pruebas unitarias para evaluar la cobertura del código.
- Revisar las recomendaciones de seguridad y confiabilidad que permanezcan pendientes.
- Verificar los problemas aceptados y falsos positivos para asegurar que sus justificaciones sean técnicamente válidas.
- Continuar realizando análisis estáticos después de futuras modificaciones.

Además, SonarQube Community Build presenta limitaciones en determinados análisis de seguridad avanzados, por lo que los resultados no deben interpretarse como una garantía de ausencia de vulnerabilidades.

6 EL BACKLOG DE REFACTORIZACIÓN CON LOS DEFECTOS QUE SOLUCIONARON

Tras el análisis inicial con SonarQube, se identificaron 8 problemas (3 de seguridad, 5 de confiabilidad y 1 de mantenibilidad). Para gestionar su resolución, nuestro equipo creó un Backlog de Refactorización, priorizando los defectos según su severidad y riesgo para el sistema.

A continuación, se detalla el backlog con las tareas ejecutadas, sus criterios de aceptación y el estado de cierre de cada ítem:

Tabla 5

"Backlog de Refactorización y Cierre de Defectos"

ID	Defecto Detectado (SonarQube)	Archivo / Componente	Severidad	Tarea de Refactorización	Criterio de Aceptación	Estado
BL-01	Ejecución síncrona del servidor (Code Smell)	Program.cs	Mayor	Reemplazar app.Run() por await app.RunAsync().	El proyecto compila y SonarQube elimina la advertencia de asincronía.	**Cerrado**
BL-02	Falta de encabezado de seguridad HTTP (Vulnerabilidad)	web.config	**Crítica**	Incorporar el encabezado X-Content-Type-Options con valor nosniff.	El navegador no interpreta tipos MIME incorrectos. Alerta de seguridad mitigada.	**Cerrado**
BL-03	Recursos externos sin integridad (Vulnerabilidad)	_Layout.cshtml	**Crítica**	Agregar atributos integrity y crossorigin (SRI) a Bootstrap Icons.	El recurso externo verifica su hash criptográfico	**Cerrado**

					antes de cargar.	
BL-04	Problema de accesibilidad en formulario (Code Smell)	Login.cshtml	Menor	Eliminar el atributo autofocus.	El formulario cumple con las reglas de accesibilidad de SonarQube.	**Cerrado**
BL-05	Etiquetas no asociadas a campos de entrada (Code Smell)	GestionReservas/ Index.cshtml	Mayor	Vincular <label> con su <input> usando el atributo for e id.	Lectores de pantalla identifican correctamente el campo de justificación.	**Cerrado**
BL-06	Atributo HTML duplicado (Bug)	_Layout.cshtml	Menor	Eliminar la declaración duplicada del atributo asp-area.	El enlace de navegación funciona correctamente sin código redundante.	**Cerrado**
BL-07	Advertencia en modelo de datos (Falso Positivo)	Equipo.cs	Info	Revisión manual del modelo y gestión como "Falso Positivo" en SonarQube.	Se confirma que Entity Framework requiere el tipo de dato.	**Cerrado**

Evidencias y seguimiento

El seguimiento de estas tareas se realizó en conjunto con las herramientas de desarrollo. Tras aplicar las correcciones correspondientes a los ítems BL-01 al BL-06, se ejecutó una nueva compilación (dotnet build) y un nuevo escaneo con SonarScanner.

Como se evidencia en la Figura 17 (Comparación visual), el resultado del segundo análisis arrojó 0 nuevos problemas, manteniendo la Puerta de Calidad Aprobada. Esto demuestra que el backlog fue resuelto de manera lógica, mejorando la seguridad (mitigando las 3 vulnerabilidades iniciales) y la mantenibilidad del código base para LabTrack sin romper la arquitectura existente.